# CprE 3810: Computer Organization and Assembly-Level Programming

# Project Part 1 Report

Team Members: Ryan Hoefer_____

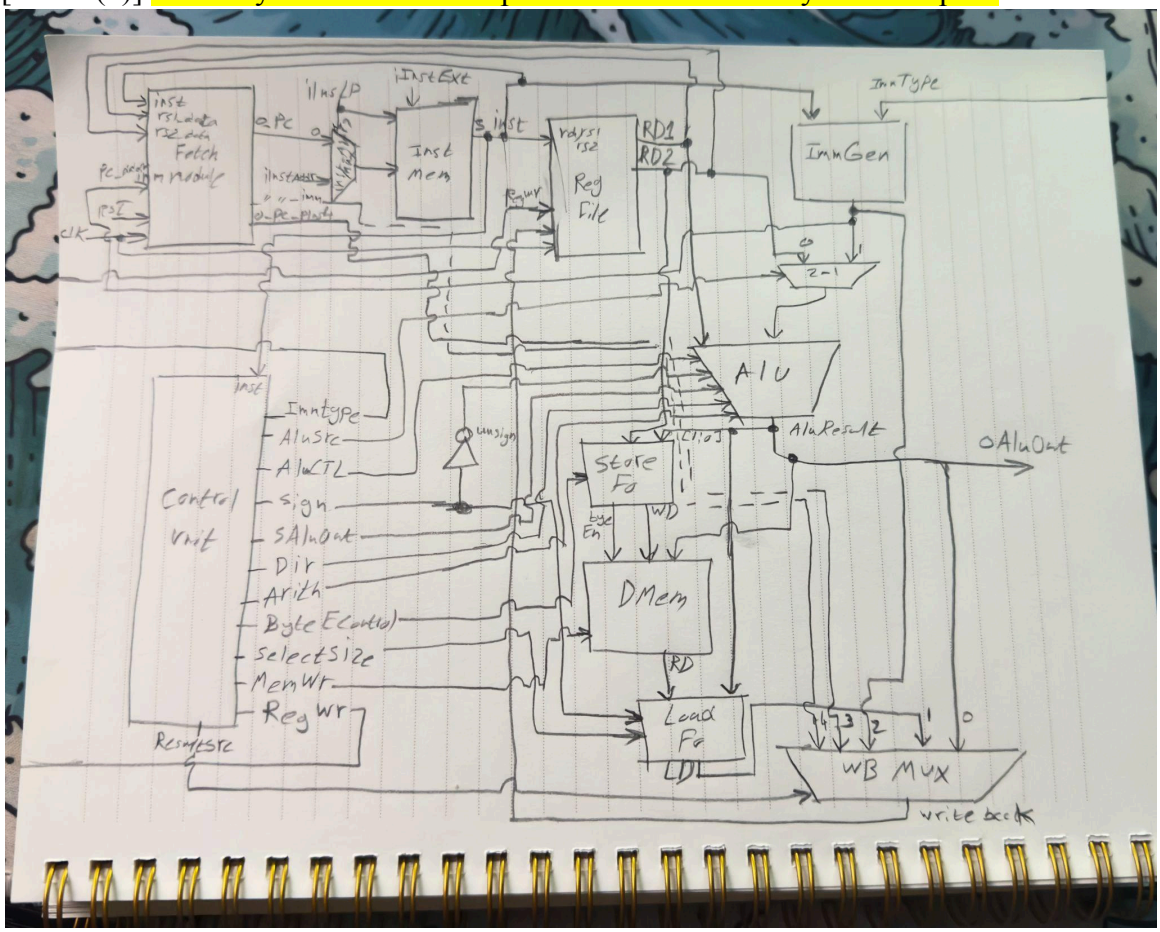
Tariq Almaaini_____

_____

Project Teams Group #:_F_02_____

***Refer to the highlighted language in the project 1 instruction for the context of the following questions.***

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.

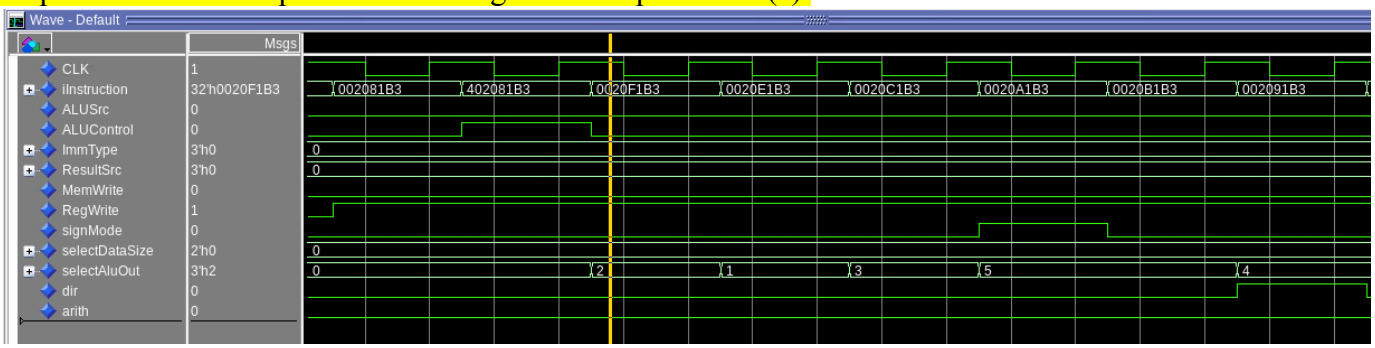



[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the

*N* control signals needed by your datapath implementation. The end result should be an *N*M* table where each row corresponds to the output of the control logic module for a given instruction

- The attached spreadsheet shows the instructions supported by our RISCV processor, along with their opcodes, funct fields, and corresponding control signals. Each row represents an instruction and the control signals generated by our Control Unit. Branch decisions are handled separately by the Fetch Module.

[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).




These are for the instructions from left to right

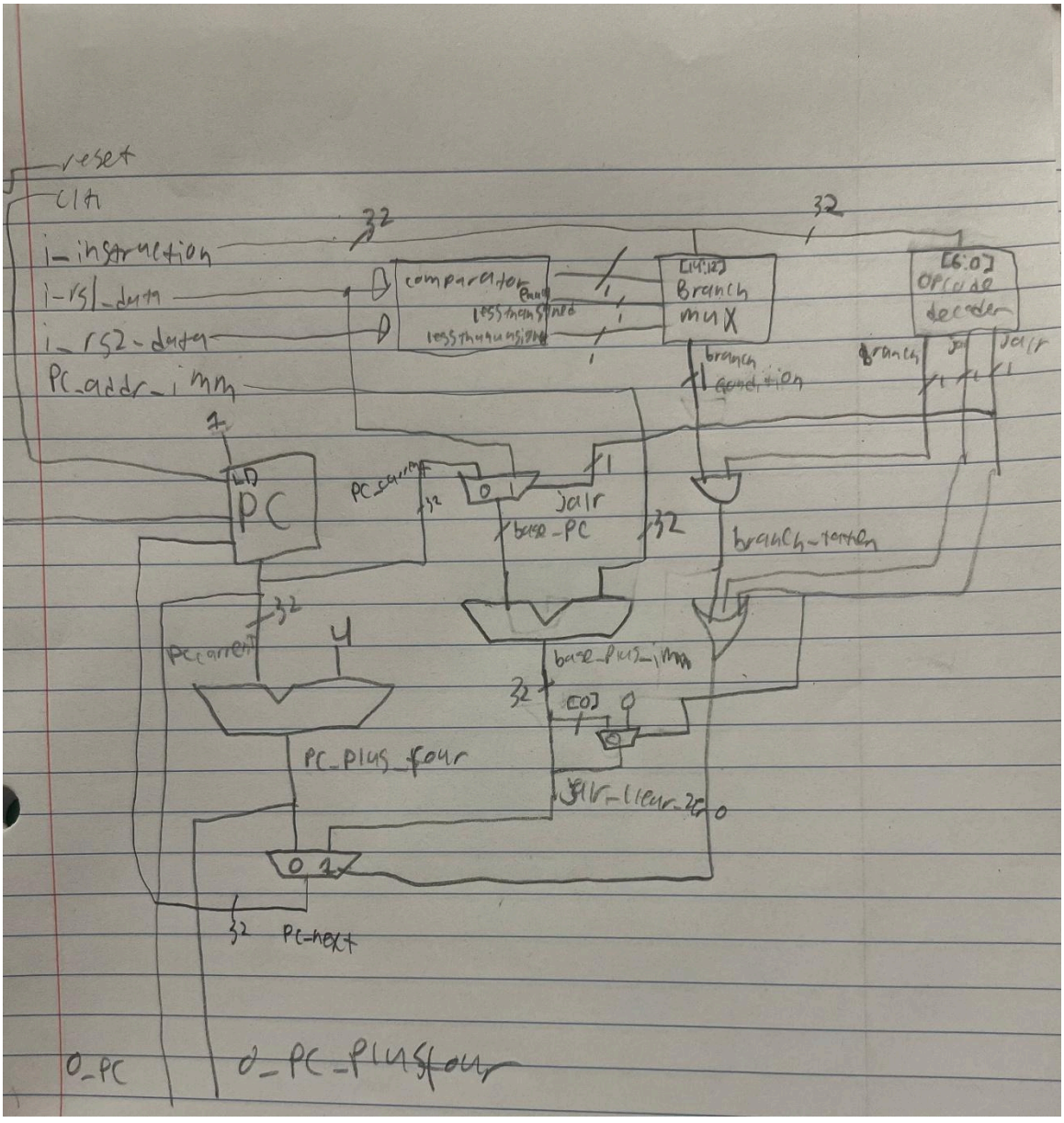
1: ADD - only reg write is needed
2: SUB - alu control is needed because we are subtracting
3: AND - alu out is set to 2 because we want to select the out
4: OR - alu out is 3 because select the or output
5: XOR - alu out is 4 because select the xor output
6: SLT - alu out is 5 to select from the alu output mux, sign mode is 1 because it is a signed operation
7: SLTU - sign mode is changed to 0 because it is unsigned
8: SLL- ALU output is 4 to select from the barrel shifter, dir is 1 becasue it is left,

Regwrite is needed for all above because they all write back

[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.
Normal instructions would add +4 to the PC – eg: add, addi, lw,
instructions that would work differently would be beq, bge, bgeu, blt, bltu, bne, jal, jalr

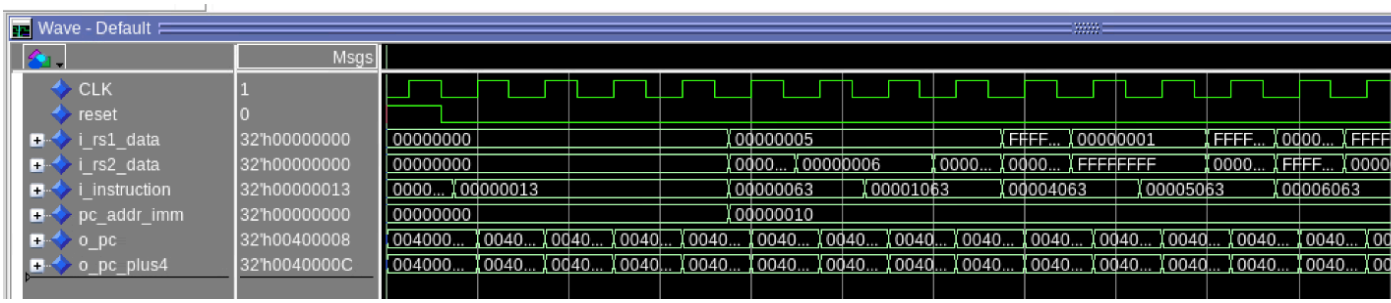
need a PcSrc control signal to decide whether to set the PC to PC + 4 or use PC +{Imm,1b'0} if the branch comparison is true, or for jalr set the PC to (rs1 + immediate) & 0xFFFFFFFE. For Jal and Jalr, write current PC+4 into rd

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?




The additional control signals that are needed are the branch, jal, and jalr signals to specify what type of jump in the pc would happen.

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.




- We implemented the instruction fetch logic using VHDL and tested it in questasim. We made a testbench with 25 different tests to check the PC updates, branch instructions, JAL, JALR, and reset. As shown in the waveform, the PC start at 0x00400000 and increases by 4 during normal execution. For the branch instruction, we tested what happens when the condition is true and false. If the branch is taken, the PC jumps to the target address, else, it just increases by 4. We also tested JAL with forward and backward jumps and JALR with different register values. We checked that JALR clears the last bit of the target address. Finally we tested reset to make sure the PC go back to 0x00400000. All 25 tests passed in questasim without any errors showing that our fetch logic works as expected for the tested cases.

[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?

Srl will prepend zeros when it shifts right, but sra will prepend the sign bit when it shifts. You dont need sla because appending the sign bit when you shift doesn't really make sense, and serve any real functional purpose.

[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.
My code uses a mux to select the extend bit, so it will be 0 for logical, and be the sign bit for arithmetic instructions. It selects based on an i_arithmetic control input into the barrel shifter that will be generated by the control unit.

[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations. You can reverse the input, shift it accordingly, then reverse it back to get a left shift.

- We can support left shifting by reversing the input bits, shifting them using the right barrel shifter, and then reversing the output bits. This allows us to perform both shifts using the same barrel shifter.

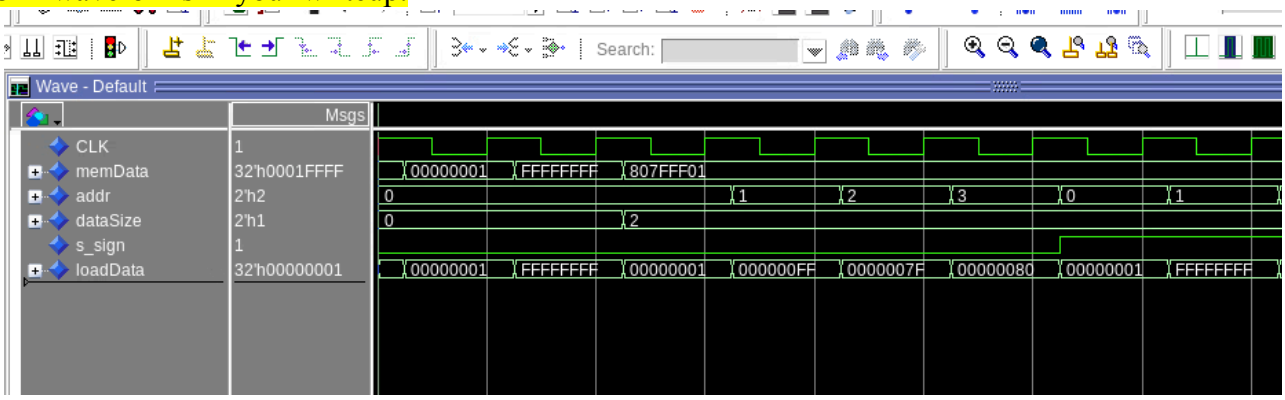
[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.

- The questasim waveforms show how the barrel shifter performs SLL, SRL, and SRA. For SLL, the bits shift left and zeros are added on the right. For SRL, the bits shift right and zeros are added on the left. For SRA, the bits shift right while keeping the sign bit. The output changes based on 1- the shift direction 2- shift amount 3- operation selected.

[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

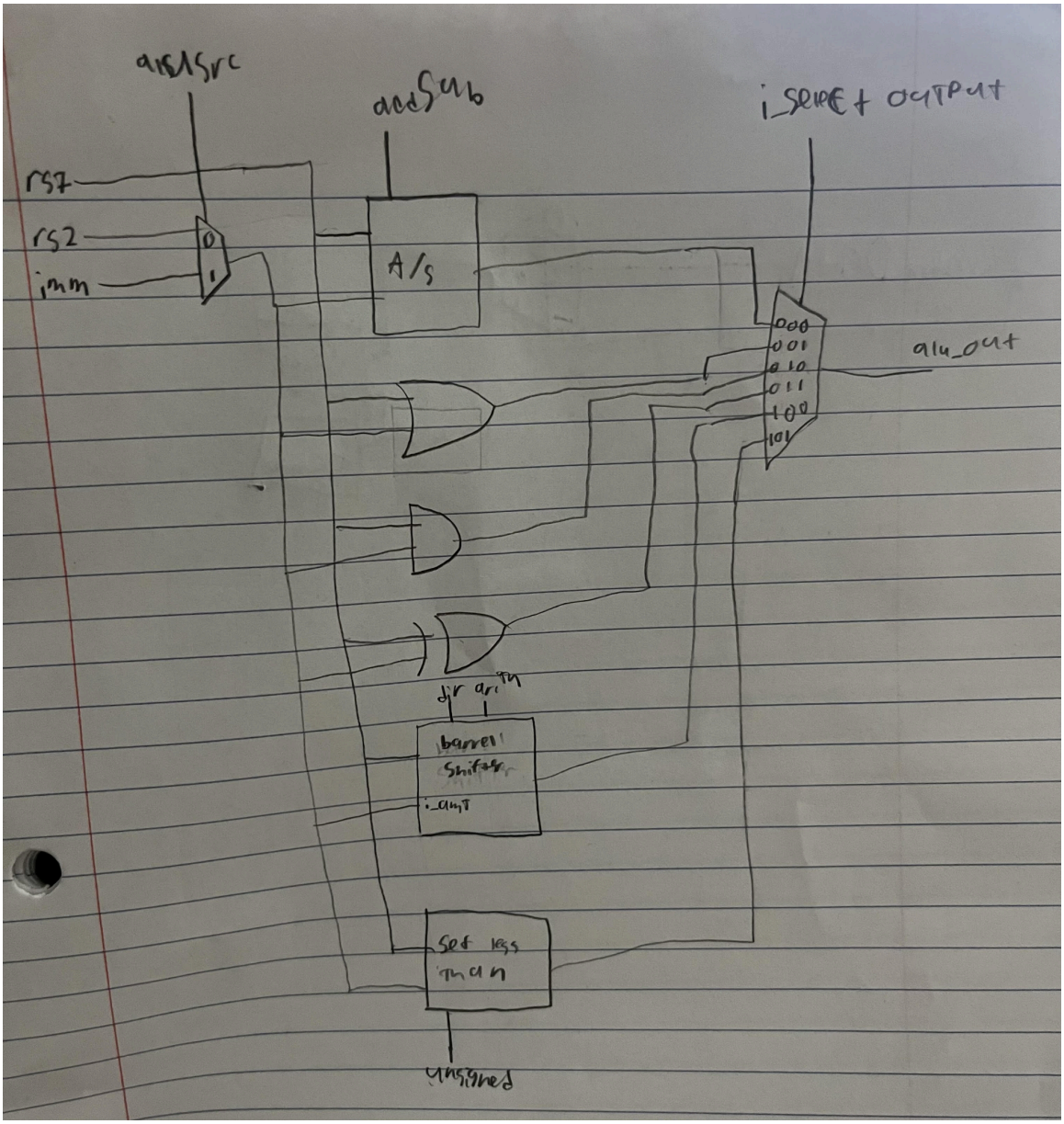
To implement the lb and lh functionality, our design first selects the byte or halfword from memory, then extends it accordingly. The design decision we made was that instead of having a large writeback mux, we would select which size of output we would need before we get to the writeback mux. So the control would produce a select signal to select either a byte, hlfword or a full word. Then that selected output is passed to the writeback mux.

[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.



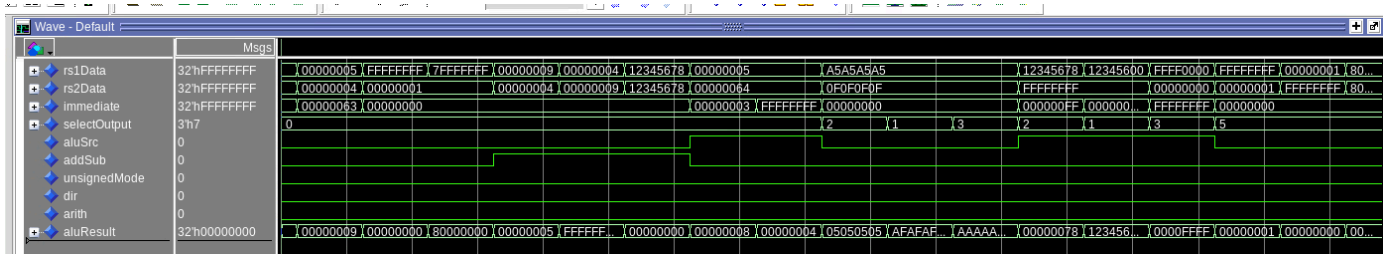

For the first two, the datasize si 0 si it should pass all the data through.
For the third, datasize is 2 so it is expecting bytes and it is byte zero from the address unsigned.
For the fourth, it is byte again but the second byte.
For the fifth, and sixth, they are the third and fourth byte respectively.
And for the seventh test, it is sign extended so the data is still outputting 1

But for the
[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?



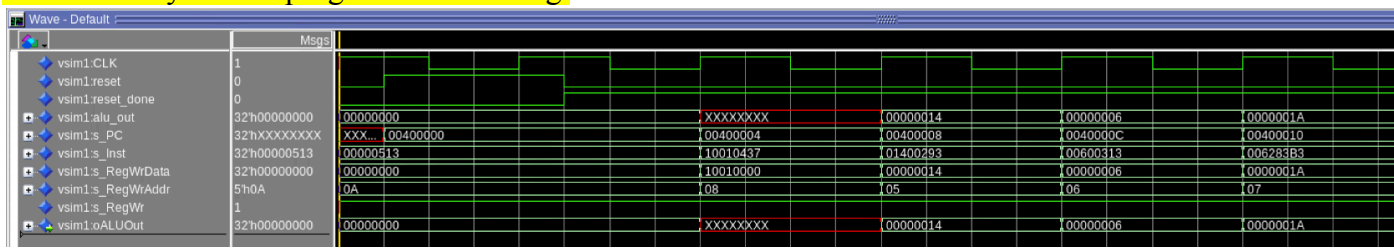

Slt is implemented by comparing rs1 with rs2 and setting the LSB of the output with 1 if rs1 < rs2 or 0 if rs2 >= rs1. Zero is not calculated in the ALU but is instead the branch calculation logic is done in the fetch unit. That is done using dataflow comparisons, for example, if rs1 < rs2 for instruction blt, then the branch would be taken.

[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

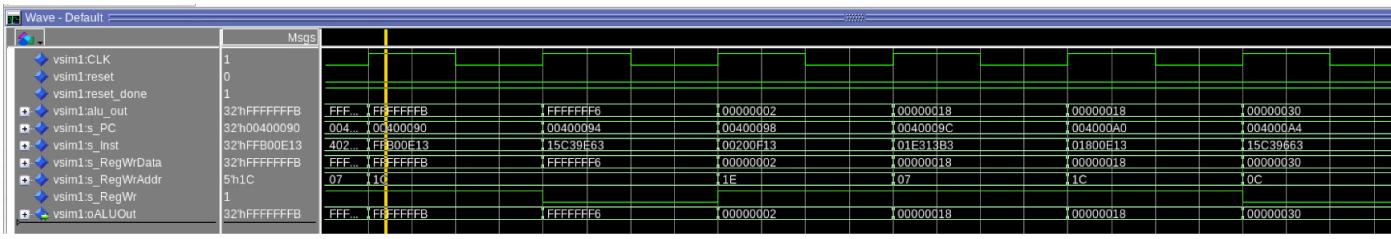



From left to right
1: Add 5 + 4 = 92
2: -1 + 1 = 0
3: adding 1 to rs1 should increment the leftmost bit by 1
4: add sub should be 1 to subtract then 9 - 4 = 5
5: addSub should still be 1 and 4 - 9 is -5
6: when subtracting, a number by itself should be 0
7: alusrc is 1 so it will use the immediate value to add rs1 + imm. Rs2 is ignored as it should
8: the sign extended immediate is added to rs1 to create -4
9: AND - selecting 2 as the output is and which is reflected in the output
10: OR - selectOutput being 1 selects OR for aluResult
11: xor - selectOutput being 3 selects xor for aluResult
12: ANDi - alusrc is 1 so we are ising the immediate values, 0x78 is preserved from the immediate being all 1's
13: ORi - same thing as before but we are selecting the or output
14: XORi - The bits in rs1 gets flipped confirming an xor as the xor is selected for the output
15: slt - since slt is selected with a value of 5, and -1 < 1, aluResult is set to 1
16: slt - since slt is again selected, but this time 1 !< -1 so aluResult is set to 0

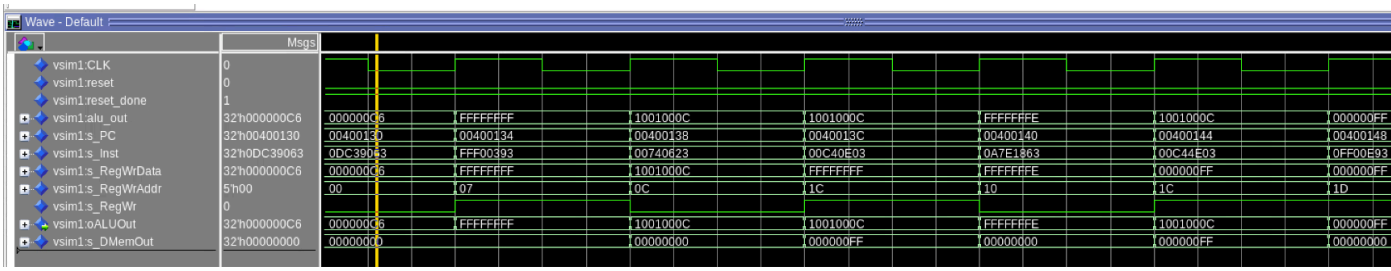
[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.



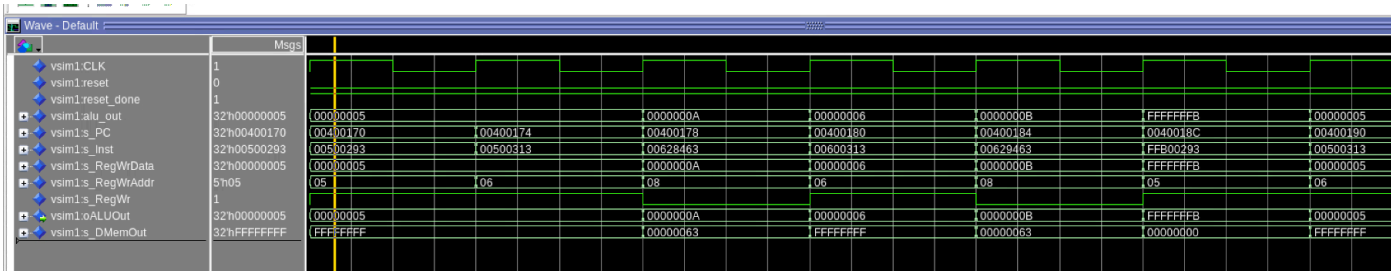

- This screenshot shows the waveform for executing ADDI and ADD instructions. The processor loads 20 and 6 into registers and adds them to get 26.



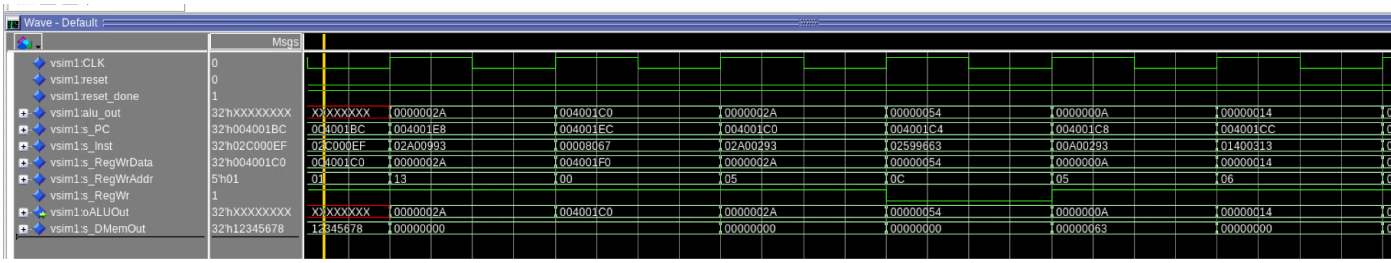

- Screenshot showing the shift instructions working correctly. The processor performs left, right, and arithmetic shifts and produces the expected results




- screnshot showing the memory load instructions. The processor correctly handles signed and unsigned byte loads and produces the expected register values.




- screenshot showing the branch instructions. The PC jumps to the target address when a branch is taken and continues normally when it is not taken.

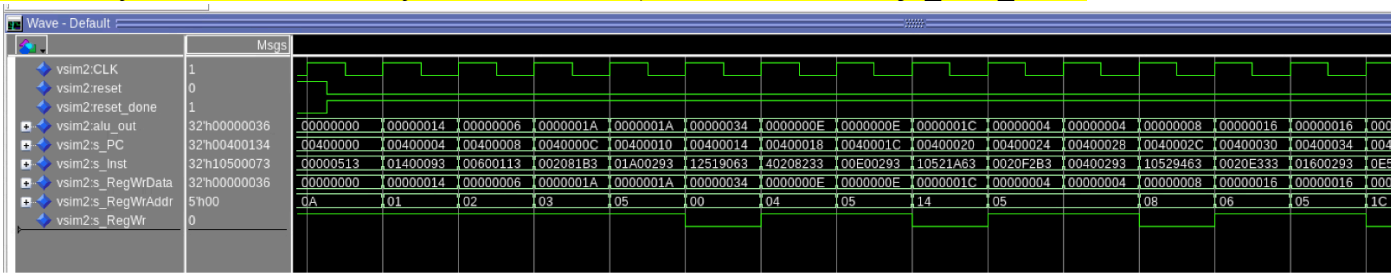



- Questasim waveform showing JAL and JALR instructions. The processor jumps to the function, sets s3 to 42, and returns to the correct address to continue execution.
- Our test plan is comprehensive because we tested the different types of instructions supported by our processor, including arithmetic, logical, shifting, memory, and control flow instructions. We also tested different conditions to

make sure the processor handles them correctly. The QuestaSim waveforms show the expected results, and our test program passed all 119 executed instructions

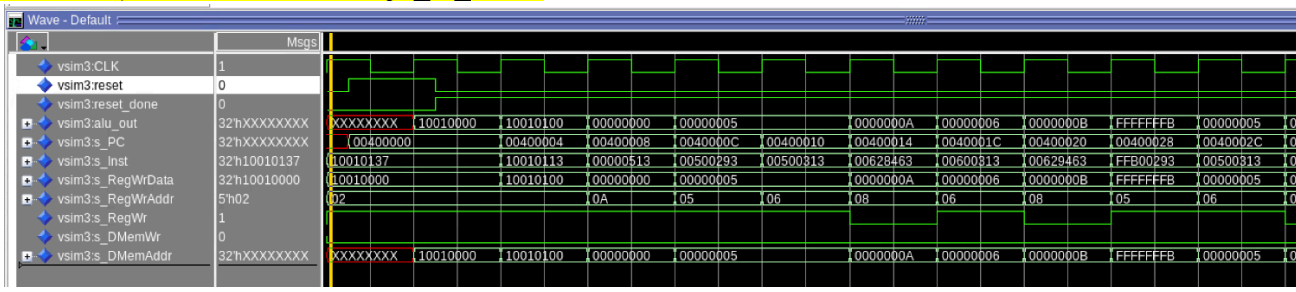
[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.

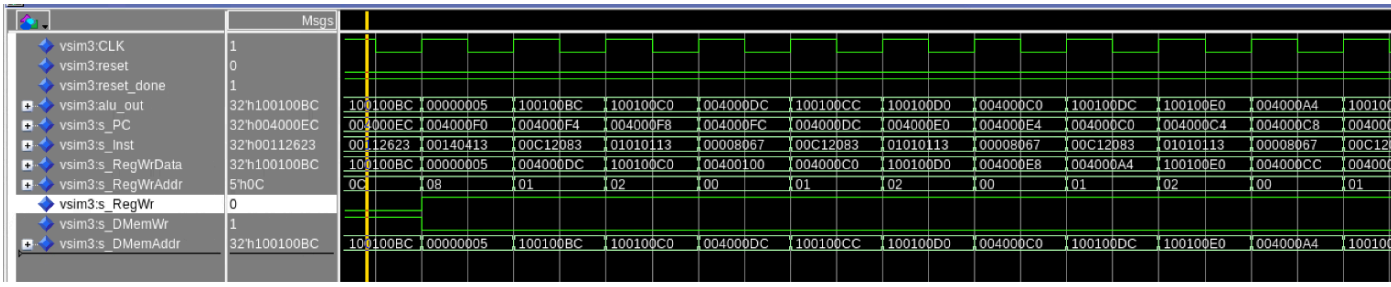



- We created a test program to check all the required arithmetic and logical instructions. The waveform shows the processor executing instructions and writing the correct results into registers. For example, ADD gives 26, SUB gives 14, and AND gives 4. We also tested register dependencies to make sure previous results can be used by later instructions. The program passed in both RARS and questasim

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.



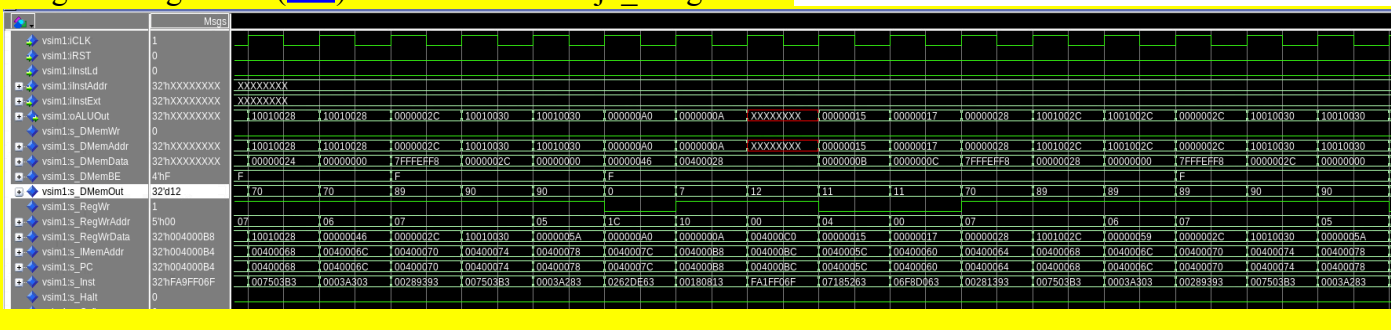

- This screenshot shows the branch instructions. The PC jumps to the correct target addresses when the branch conditions are met, and skipping the failure instruction




- This screenshot shows th processor executing nested function calls using JAL. It also shows the return addresses being saved on the stack using SW. the PC changes to the correct function addresses as each function is called

- We created a control flow test program that includes all six branch instructions, jal, and jalr. The waveforms show the pc jumping to the correct addresses and the processor executing five nested function calls. The program passed both in Rars and qustasim.

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.




This shows the tail end of mergesort reading values from memory during its final merge pass. The values are compared without being rewritten indicating it is sorted.

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

The maximum frequency my processor can run at is 55.747 ns.

```
105      55.747      0.104 FF  CELL  register_file:registerFile|register_N:\G_32_reg:6:reg32|dffg:\G_reg_N_Bit:24:Gdff|s_Q
106  Data Required Path:
107  Total (ns)  Incr (ns)     Type  Element
108  ==========  ========= ==  ====  ===================================
109      20.000     20.000           latch edge time
110      23.426      3.426  R        clock network delay
111      23.434      0.008           clock pessimism removed
112      23.414     -0.020           clock uncertainty
113      23.432      0.018     uTsu  register_file:registerFile|register_N:\G_32_reg:6:reg32|dffg:\G_reg_N_Bit:24:Gdff|s_Q
114  Data Arrival Time  :    55.747
115  Data Required Time :    23.432
116  Slack              :   -32.315 (VIOLATED)
117  ================================================================================
```

Some components that could be improved upon is the ALU, Immediate generator and the the memory units as they take the longest wit the ALU being about 6.5 ns,

The immediate generator being about 4 ns, and the memory units being about 16ns each

